IMT Atlantique
Projet 28
Campus de Brest

UI AI Natives – Réinventer l'interaction homme-machine à travers les Generative UIs

Architecture du Projet

Version 1

Date 17/10/2026

Auteurs :
BADSSI Chaimae
BARET Alexis
BEDOUIN Arnaud
BELKHIRIA Zeineb
MOREAU Maxandre
SAFTA Yassine

Encadré par : STOENESCU Daniel

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Table des matières

1 Vue d'ensemble **2**

2 Organisation du dépôt **3**

3 Composants **4**

3.1 Frontend 4

3.2 Agent 4

3.3 Backend 5

3.4 Données 5

4 Contrats d'interface **6**

4.1 Page – runtime – agent : AG-UI 6

4.2 Agent – backend : outils MCP 6

4.3 Page – backend : API HTTP 6

4.4 Agent – page : catalogue A2UI 7

5 Déroulement des cas d'usage **7**

5.1 Consultation et analyse 7

5.2 Virement 8

5.3 Produits et conseiller 8

1 Vue d'ensemble

L'application se compose de quatre programmes et d'une base de données hébergée sur Supabase (figure 1). Le navigateur n'exécute que la page ; les programmes tournent côté serveur, et la base est partagée par toute l'équipe.

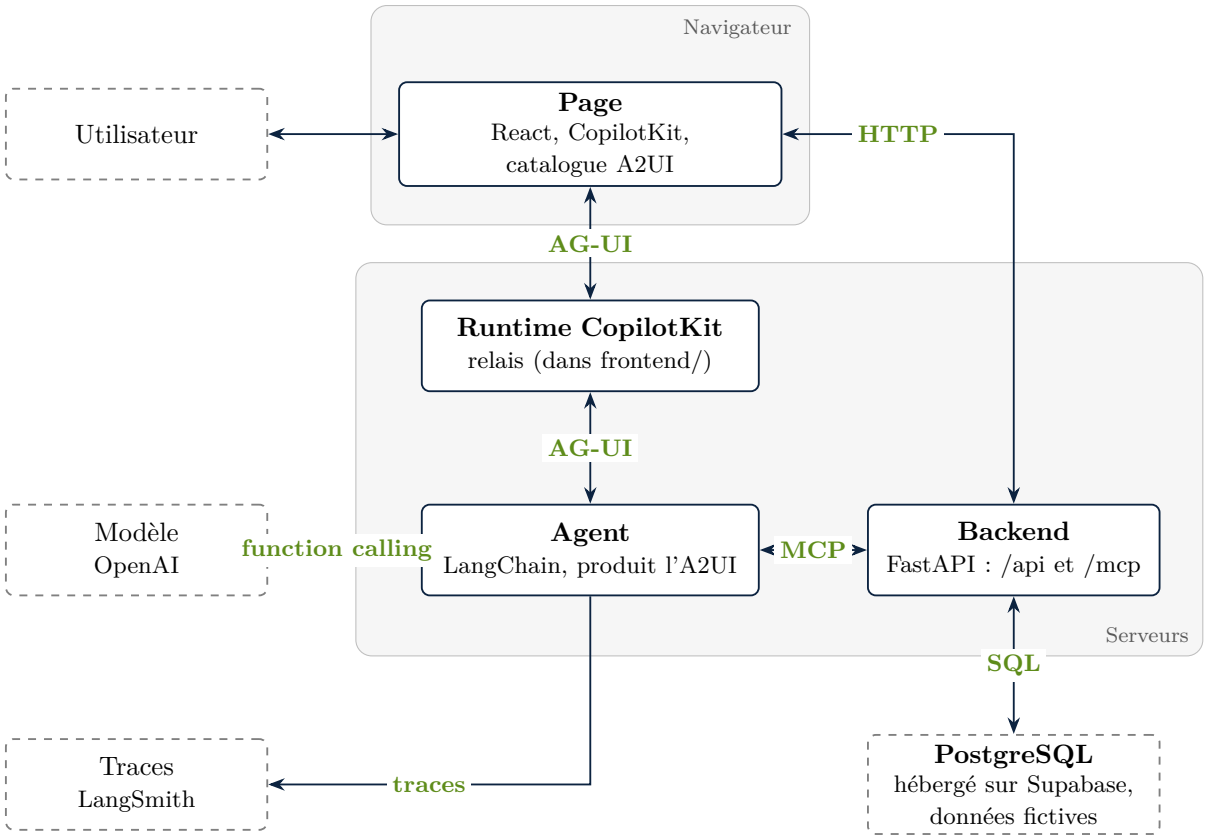


FIGURE 1 – Vue d'ensemble. Les cadres en tirets sont des services extérieurs. Le lien HTTP direct entre la page et le backend ne sert qu'à la connexion et à la confirmation des virements.

Programme	Dossier	Port	Rôle
Page	`frontend/`	5173	Connexion, conversation, affichage des composants.
Runtime CopilotKit	`frontend/`	4000	Relais AG-UI entre la page et l'agent, traitement de l'A2UI.
Agent	`agents/agent_01/`	8001	Dialogue avec le modèle, appel des outils, description des interfaces.
Backend	`backend/`	8000	API bancaire (`/api`) et outils de l'agent (`/mcp`).
PostgreSQL	– (Supabase)	–	Base hébergée en ligne, partagée par l'équipe ; données fictives.

TABLE 1 – Programmes et ports en développement.

Quatre principes guident cette répartition :

1. **L'agent ne touche jamais la base de données.** Il n'agit que par les outils MCP du

backend, qui appliquent les mêmes contrôles que l'API.

2. **Les règles métier sont écrites une seule fois**, dans la couche `services/` du backend, partagée par les routes `/api` et par les outils `/mcp`.
3. **L'identité vient de la session, jamais du modèle.** Le client concerné par une requête est déduit du jeton de connexion ; aucun outil ne prend d'identifiant de client en paramètre.
4. **Le modèle ne peut pas exécuter de virement.** Il ne peut que préparer une simulation ; la confirmation passe directement de la page au backend, avec le mot de passe de l'utilisateur.

2 Organisation du dépôt

Les noms de fichiers, de dossiers, de routes, d'outils et de composants sont en anglais. Les éléments déjà présents dans le dépôt sont marqués « existant » ; les autres sont à créer.

```
generative-banking-ui/
|-- frontend/                      React + Vite + CopilotKit (existant)
|   |-- src/
|   |   |-- main.tsx               CopilotKitProvider (existant)
|   |   |-- App.tsx                navigation: login, then assistant (existant)
|   |   |-- vite-env.d.ts          types of the VITE_ variables (existant)
|   |   |-- pages/
|   |   |   |-- Login.tsx          username and password
|   |   |   `-- Assistant.tsx      conversation (CopilotChat)
|   |   |-- catalog/               A2UI banking components
|   |   |   |-- AccountCard.tsx
|   |   |   |-- TransactionList.tsx
|   |   |   |-- SpendingChart.tsx
|   |   |   |-- TransferConfirmation.tsx
|   |   |   |-- ProductCard.tsx
|   |   |   |-- AppointmentPicker.tsx
|   |   |   `-- index.ts           catalog declaration
|   |   `-- api/
|   |       |-- auth.ts            login, logout
|   |       `-- transfers.ts       transfer confirmation
|   |-- runtime.ts                 CopilotKit runtime (existant)
|   |-- index.html                 (existant)
|   |-- vite.config.ts             (existant)
|   |-- package.json               (existant)
|   `-- .env.example               (existant)
|-- agents/                        one folder per agent
|   `-- agent_01/                  banking assistant, LangChain (Python)
|       |-- main.py                FastAPI app + AG-UI endpoint
|       |-- agent.py               model, prompt, middlewares (existant)
|       |-- prompt.md              system prompt (existant)
|       |-- tools.py               MCP client to the backend
|       |-- ui/                    A2UI surfaces
|       |-- tests/
|       |-- requirements.txt
|       `-- .env.example
|-- backend/                       FastAPI + PostgreSQL (existant)
|   |-- app/
|   |   |-- main.py                mounts /api and /mcp (existant, vide)
|   |   |-- api/                   auth.py, accounts.py, transactions.py,
|   |   |                          transfers.py, products.py, advisors.py
|   |   |-- mcp/server.py          agent tools
|   |   |-- services/              shared business logic
|   |   |-- schemas/               request and response formats
|   |   `-- security.py            password hashing, session tokens
|   |-- models/                    SQLAlchemy models (existant) + user_sessions.py,
```

```
|   |                              audit_logs.py, beneficiaries.py,
|   |                              transfer_simulations.py
|   |-- migrations/                Alembic (existant)
|   |-- database.py                (existant)
|   |-- seed_database.py           (existant)
|   |-- tests/
|   `-- .env.example
|-- database/diagrams/             UML diagram (existant)
|-- docs/                          état de l'art, architecture
|-- experiments/                   état de l'art experiments
`-- README.md
```

Chaque dossier de premier niveau correspond à un chantier indépendant, avec ses propres dépendances : il peut être confié à une partie de l'équipe sans gêner les autres, les échanges entre dossiers passant uniquement par les contrats de la section 4. Le dossier `agents/` contient un sous-dossier par agent : `agent_01` est l'assistant bancaire, et d'éventuels agents spécialisés, reliés par A2A, y prendraient place à côté.

3 Composants

3.1 Frontend

La page est une application React construite avec Vite. Elle comporte deux écrans : la **connexion**, par identifiant et mot de passe (`Login.tsx`), et l'**assistant**, qui affiche la conversation CopilotKit (`Assistant.tsx`).

— **Connexion.** La page appelle directement le backend (`POST /api/auth/login`) et conserve le jeton de session reçu. Ce jeton est ensuite joint à chaque requête, vers le backend comme vers le runtime.
— **Conversation.** Le fournisseur `CopilotKitProvider` relie la page au runtime ; le composant `CopilotChat` affiche les messages, les étapes de l'agent et les surfaces A2UI.
— **Catalogue A2UI.** Le dossier `catalog/` contient les composants bancaires que l'agent peut afficher (section 4.4) ; ce sont des composants React ordinaires, au style de l'application. Un composant absent du catalogue ne peut pas être affiché.
— **Confirmation des virements.** Le composant `TransferConfirmation` redemande le mot de passe et appelle directement le backend, sans passer par l'agent (section 5.2).

Le **runtime CopilotKit** est un petit serveur Node (`runtime.ts`, lancé par `npm run runtime`). Il déclare l'agent par son adresse AG-UI, lue dans la variable `AGENT_URL`, active le traitement de l'A2UI et transmet le jeton de session à l'agent *(à vérifier)*. Il partage le `package.json` de la page, ce qui garantit que les paquets CopilotKit de la page et du runtime ont la même version ; `@ag-ui/client` doit aussi rester à la version exacte qu'utilise CopilotKit.

3.2 Agent

L'agent (`agents/agent_01/`) est écrit en Python avec LangChain (`create_agent`) et exposé en AG-UI par un serveur FastAPI, comme dans l'exemple de l'état de l'art.

— **Modèle.** Un modèle d'OpenAI, choisi par une variable d'environnement ; LangChain permet d'en changer sans modifier le code.

— **Consigne** (`prompt.md`). Rôle d'assistant bancaire, interdiction d'inventer des données, obligation de passer par les outils et d'afficher les résultats avec les composants du catalogue plutôt qu'en texte.
— **Outils** (`tools.py`). L'agent ne définit pas d'outil métier : il se connecte au serveur MCP du backend au démarrage et en découvre la liste (`langchain-mcp-adapters`), en lui transmettant le jeton de l'utilisateur.
— **Interfaces** (`ui/`). Construction des surfaces A2UI : création de la surface, composants du catalogue et données à afficher.
— **Observabilité.** Les traces sont envoyées à LangSmith (offre gratuite, compte unique de l'équipe), ce qui ne pose pas de problème avec des données fictives.

3.3 Backend

Le backend existant (FastAPI, SQLAlchemy, Alembic) devient le point d'entrée unique vers les données. Son application `app/main.py` monte deux parties :

— `/api` : les routes HTTP utilisées par la page (connexion, confirmation de virement) et utiles aux tests ;
— `/mcp` : le serveur MCP (`app/mcp/server.py`), en transport HTTP, qui expose les outils de l'agent.

Les deux appellent la même couche `services/`, qui contient la logique métier (calcul des dépenses par catégorie, préparation d'un virement…) et vérifie systématiquement que les données demandées appartiennent au client connecté. Le module `security.py` gère le hachage et la vérification des mots de passe, le blocage temporaire après plusieurs échecs et les jetons de session.

3.4 Données

La base PostgreSQL est hébergée sur Supabase et déjà remplie avec les données fictives. Elle est partagée par toute l'équipe : une migration Alembic modifie les tables pour tout le monde, et n'est donc lancée que par la personne chargée du backend, après relecture. Supabase n'est utilisé que comme hébergeur de PostgreSQL ; le backend y accède avec SQLAlchemy, comme à n'importe quelle base PostgreSQL.

Les modèles existants couvrent les clients, conseillers, utilisateurs, comptes, transactions, catégories, cartes, produits et souscriptions. Le modèle `User` stocke aujourd'hui un code PIN haché (`app_pin_hash`, avec un compteur d'échecs et une date de blocage) : la connexion se faisant par mot de passe, ce champ est à remplacer par un mot de passe haché, par une migration Alembic. Quatre modèles prévus par le diagramme UML restent à créer :

Modèle	Fichier	Usage dans l'architecture
`UserSession`	`user_sessions.py`	Jetons de session émis à la connexion (empreinte du jeton, expiration, révocation).
`AuditLog`	`audit_logs.py`	Journal des actions : connexions, appels d'outils, confirmations de virement.
`Beneficiary`	`beneficiaries.py`	Bénéficiaires enregistrés par le client, seuls destinataires possibles d'un virement.
`TransferSimulation`	`transfer_simulations.py`	Virement préparé par l'agent, en attente de confirmation (statut, date de confirmation).

TABLE 2 – Modèles à ajouter dans `backend/models/`.

4 Contrats d'interface

4.1 Page – runtime – agent : AG-UI

La page et l'agent échangent des événements AG-UI, relayés par le runtime : message de l'utilisateur et historique dans un sens ; texte en streaming, étapes, appels d'outils et surfaces A2UI dans l'autre. Le jeton de session accompagne chaque requête dans l'en-tête `Authorization`.

4.2 Agent – backend : outils MCP

Les outils sont déclarés dans `backend/app/mcp/server.py`. Aucun ne prend d'identifiant de client : le backend le déduit du jeton transmis par l'agent.

Outil	Paramètres	Renvoie	Accès
`list_accounts`	–	Comptes du client, avec solde	Lecture
`get_recent_transactions`	compte, nombre	Dernières opérations d'un compte	Lecture
`get_spending_by_category`	période	Total des dépenses par catégorie	Lecture
`compare_spending`	deux périodes	Écart des dépenses par catégorie	Lecture
`list_beneficiaries`	–	Bénéficiaires enregistrés	Lecture
`prepare_transfer`	compte source, bénéficiaire, montant, motif	Simulation en attente et son récapitulatif	Écriture (simulation)
`list_products`	type (facultatif)	Produits bancaires disponibles	Lecture
`get_product_details`	produit	Description et tarif d'un produit	Lecture
`get_advisor`	–	Conseiller attribué au client	Lecture
`get_appointment_slots`	motif	Créneaux de rendez-vous simulés	Lecture

TABLE 3 – Outils MCP exposés à l'agent.

4.3 Page – backend : API HTTP

La page n'appelle le backend directement que pour deux opérations, qui ne doivent pas dépendre du modèle :

Route	Rôle
`POST /api/auth/login`	Vérifie l'identifiant et le mot de passe, renvoie un jeton de session.
`POST /api/auth/logout`	Révoque le jeton.
`POST /api/transfers/{id}/confirm`	Vérifie le mot de passe, puis confirme la simulation si elle appartient au client et est encore en attente.
`POST /api/transfers/{id}/cancel`	Annule la simulation.

TABLE 4 – Routes appelées directement par la page.

Les autres données (comptes, transactions, produits) sont aussi exposées en lecture sous `/api`, en appelant les mêmes services que les outils MCP : elles servent aux tests et pourront servir à des écrans classiques.

4.4 Agent – page : catalogue A2UI

Composant	Données	Cas d'usage
`AccountCard`	nom, IBAN masqué, solde	Consultation
`TransactionList`	titre, lignes (date, libellé, montant)	Consultation, dépenses
`SpendingChart`	période, montants par catégorie	Analyse des dépenses
`TransferConfirmation`	identifiant de simulation, récapitulatif	Virement
`ProductCard`	nom, description, tarif	Produits
`AppointmentPicker`	conseiller, créneaux	Conseiller

TABLE 5 – Composants du catalogue A2UI (`frontend/src/catalog/`).

L'agent peut aussi utiliser les composants de base d'A2UI (texte, colonne, ligne, liste) pour les assembler.

5 Déroulement des cas d'usage

5.1 Consultation et analyse

Pour « Combien ai-je dépensé en restaurants ce mois-ci ? » :

1. La page envoie la question au runtime, puis à l'agent, en AG-UI, avec le jeton de session.
2. L'agent transmet au modèle la consigne, la question et la liste des outils ; le modèle demande l'outil `get_spending_by_category` pour le mois en cours.
3. L'agent appelle l'outil sur `/mcp` avec le jeton ; le backend identifie le client, calcule les dépenses et renvoie le résultat.
4. Le modèle demande l'affichage : l'agent produit une surface A2UI `SpendingChart` avec ces montants.

5. Le runtime et CopilotKit rendent la surface avec le composant du catalogue, et le texte de réponse s'affiche au fil de l'eau.

5.2 Virement

Le virement suit le même chemin jusqu'à la préparation, puis la confirmation quitte la boucle de l'agent (figure 2).

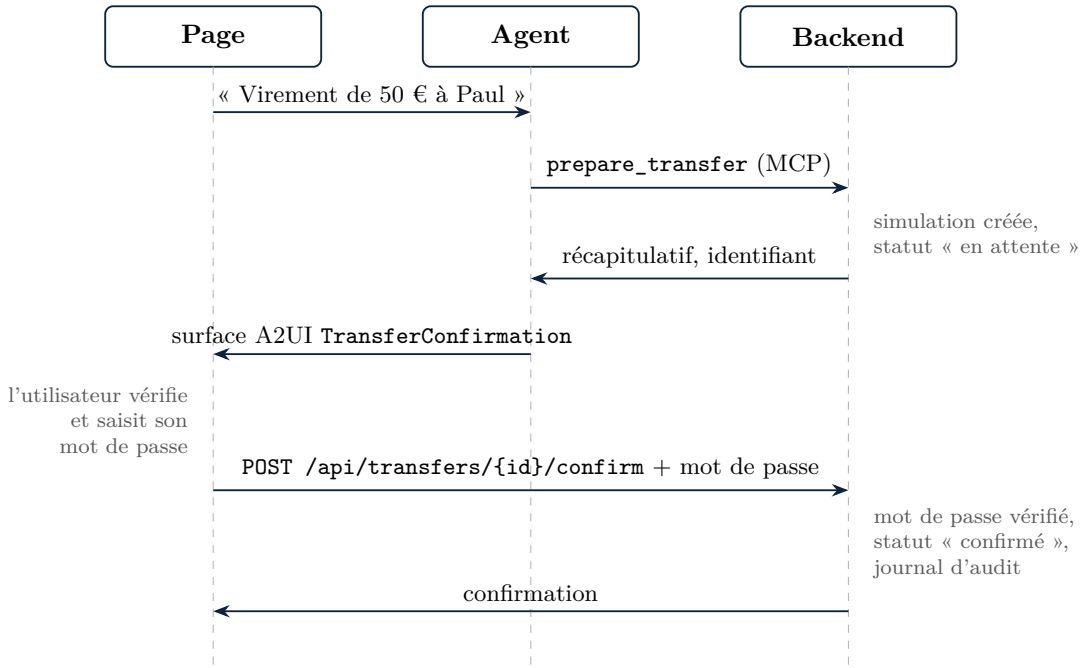


FIGURE 2 – Préparation puis confirmation d'un virement simulé. Aucun outil ne permet à l'agent de confirmer lui-même un virement.

La page peut ensuite informer l'agent du résultat, pour qu'il le mentionne dans la suite de la conversation.

5.3 Produits et conseiller

Les demandes sur les produits suivent le déroulement de la consultation, avec les outils `list_products` et `get_product_details` et le composant `ProductCard`. La demande de contact utilise `get_advisor` et `get_appointment_slots`, puis affiche `AppointmentPicker` ; la prise de rendez-vous reste simulée.